



Developer Assessment

Welkom bij het ChipSoft assessment!

Introductie & Doel

Het doel van deze assessment is eenvoudig: laat zien dat je **bestaande code kunt lezen en begrijpen**, dat je **nieuwe functionaliteiten kunt implementeren**, **bestaande issues kunt oplossen** en **goede testen kunt schrijven**. We willen jouw skills zien, niet alleen een werkende oplossing.

Context

Je werkt aan een applicatie voor artsen om hun afspraken eenvoudig in te zien en te verwerken. De applicatie moet gebruiksvriendelijk, snel en efficiënt zijn.

Details zijn minder belangrijk dan laten zien wat je allemaal in je ‘developer-toolbelt’ hebt zitten. Werk dingen grondig uit, maar verlies je dus niet in details – dat kost te veel tijd.

We verwachten dat je +- 3 à 4 uur tijd nodig hebt om de opdracht te maken, maar meer mag natuurlijk ook. Zie dit als jouw visitekaartje: overtuig ons!

Aan te leveren

Een volledig werkende build van de applicatie, zodat deze eenvoudig kan worden gestart en getest.

Minimale testdekking die aantoont dat de belangrijkste functionaliteiten zijn getest.

Duidelijke documentatie waarin je de aangebrachte wijzigingen en specifieke keuzes toelicht, inclusief eventuele instructies voor gebruik.

Beperkingen?

Eigenlijk niet! Ofja... we vragen je wél om het gebruik van AI beperkt te houden. Het gaat ons namelijk minder om de uiteindelijke code, en veel meer om jouw redenering en de keuzes die je maakt. Een prachtige codebase heeft weinig waarde als je niet kunt uitleggen waarom je het zo hebt opgezet.

Zie je hier zaken waar jij van denkt dat ze beter kunnen? Voel je dan vooral vrij om dingen aan te passen, zelfs binnen de bestaande code.

Wat er nu staat, is bedoeld als startpunt en is wat ons betreft voldoende als basis voor de gevraagde uitbreidingen. Maar als jij iets mooier of beter kunt maken, dan horen we graag waarom!

Opdracht

Breid de **BillingService** uit zodat je een factuur-object kunt aanmaken voor een patiënt (happy path).

Dek de BillingService en alle bijbehorende onderdelen volledig met unit tests.

Implementeer functionaliteit om **een afspraak gedetailleerd in te zien**. Zorg dat dit intuïtief en gebruiksvriendelijk is.

Voeg een status toe aan de afspraken en definieer daarbij logische statussen. Je mag zelf kiezen welke.

Breid de **validatie van afspraken** uit zodat:

- Afspraken niet kunnen overlappen. Veronderstel dat er genoeg locaties zijn om afspraken in te organiseren.
- Eigenschappen die mogelijks statusgebonden zijn, verplicht worden vanaf die status.

Voorzie functionaliteit om **het aanmaken van facturen** te starten en bestaande facturen in te zien.

Bonuspunten scoren?: Maak een **generieke repository** die als basis dient voor andere repositories. Dit moet een solide fundament zijn waarop verder gebouwd kan worden. We hebben de interface voorzien als leidraad, de implementatie is aan jou!

Scoring

Volledigheid: De gekozen oplossing is volledig functioneel, de applicatie werkt foutvrij en naar behoren, de nodige schermen en uitgebreide validatie zijn voorzien en alle tests slagen.

Tip: We beoordelen de oplossing op techniek en validatie. Een minimalistische interface is prima, zolang de gebruikerservaring logisch en werkbaar is.

Codekwaliteit: De code is duidelijk, compact en goed gestructureerd, met logische naamgeving en zonder overbodige elementen. De SOLID principes werden toegepast binnen de zelf voorziene structuur.

Testing: Unit tests zijn correct opgezet, voorzien van logische asserts en dekken de belangrijkste functionaliteiten.

Praktische info

Lever je oplossing in als een zip-bestand met code, tests en documentatie.

Zorg ervoor dat het zip-bestand zo klein mogelijk is door alle irrelevante bestanden uit je folder te halen.

Succes!